🧠 1. LeNet-5 (from scratch) ✅

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

class LeNet5(nn.Module):
    def __init__(self):
        super(LeNet5, self).__init__()

        self.conv1 = nn.Conv2d(1, 6, kernel_size=5)   # (1 → 6)
        self.pool = nn.AvgPool2d(2, 2)
        self.conv2 = nn.Conv2d(6, 16, kernel_size=5)  # (6 → 16)

        self.fc1 = nn.Linear(16 * 4 * 4, 120)
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, 10)

    def forward(self, x):
        x = self.pool(F.tanh(self.conv1(x)))
        x = self.pool(F.tanh(self.conv2(x)))
        x = x.view(-1, 16 * 4 * 4)
        x = F.tanh(self.fc1(x))
        x = F.tanh(self.fc2(x))
        x = self.fc3(x)
        return x
```

🚀 2. VGG16 (pretrained) ✅

```python
import torchvision.models as models

model = models.vgg16(pretrained=True)

# Modify for your dataset (e.g., 2 classes)
model.classifier[6] = nn.Linear(4096, 2)
```

🧮 3. MobileNet (lightweight) ✅

```
model = models.mobilenet_v2(pretrained=True)

# Modify last layer
model.classifier[1] = nn.Linear(model.last_channel, 2)
```

⚡ 4. EfficientNet (modern) ✅

```
model = models.efficientnet_b0(pretrained=True)

# Modify last layer
model.classifier[1] = nn.Linear(model.classifier[1].in_features, 2)
```

🔧 Important (common for all pretrained models)

Freeze layers (optional)

```
for param in model.parameters():
    param.requires_grad = False
```

Train only last layer

```
for param in model.classifier.parameters():
    param.requires_grad = True
```

🎯 What to tell students

- LeNet → built from scratch (learning)
- VGG / MobileNet / EfficientNet → used via pretrained models (real-world)